WEEK 05 · BLOCK 5B

Block 5b — Objects, Arrays, Closures & Modules

Goal: put JavaScript's structures to work — an object as a dictionary, dynamic arrays with `map` / `filter` / `reduce`, a closure with private state, and a `class` living in its own module.

Time: ~45 minutes · **You need:** a terminal with Node (from Block 0)

Setup

```
cd exercises/week05/5b/starter
node main.js        # or: npm start
```

`main.js` imports the `Stack` class from `stack.js` — two files, one program. Edit either, save, and re-run. (The `"type": "module"` in `package.json` is what lets Node use `import` / `export`.)

Task 1 — Object as a dictionary

Count how many times each word appears in `words`, using a plain object `{ }` keyed by the word. Log the tally — you should get `{ red: 3, blue: 2, green: 1 }`.

> Reach a count with `tally[word]`; remember a missing key is `undefined` — handle the first time you see a word.

Task 2 — Dynamic arrays + transforms

Starting from `nums`:

- **2a** — `push` 7 and 8, then log the array.
- **2b** — `doubled` = every number × 2 → `map`
- **2c** — `evens` = only even numbers → `filter`
- **2d** — `total` = the sum → `reduce`

Each of these takes a **function** — the first-class functions from Block 5a in action.

Task 3 — A closure

Write `makeCounter()` that **returns a function**. Each call increments and returns a count that lives *inside* the closure:

```
const next = makeCounter();
next(); // 1
next(); // 2
```

Then show there's **no way** to read or set the count except by calling `next()` — that's the private state a closure gives you.

Task 4 — The Stack module

Finish the four methods in **`stack.js`** (`push` , `pop` , `peek` , and the `size` getter) using the private `#items` array. Back in `main.js` , the provided calls should print:

```
size: 2
pop : b
peek: a
size: 1
```

You just split a class into its own module and imported it — the pattern for every real app from Block 7 on.

Requirements checklist

- [ ] Task 1 prints `{ red: 3, blue: 2, green: 1 }`
- [ ] Task 2: array grows to include 7 & 8; correct `doubled` , `evens` , `total`
- [ ] Task 3: `makeCounter` returns a working counter with **private** state
- [ ] Task 4: `stack.js` methods implemented; `main.js` prints `2, b, a, 1`

Stretch goals (optional)

- Make the word tally case-insensitive, and also log the **most frequent** word.
- Add an `isEmpty()` method to `Stack` , and a `toArray()` that returns a **copy** of `#items` (not the private array itself — why does that matter?).
- Move `makeCounter` into its own `counter.js` module and `import` it into `main.js` .